

GRADO

SERIE GUÍAS · TECNOLOGÍA E INFORMÁTICA



8°

Depuración de lógica de control: casos esperados, contrarios y límites

Guía 17-8-TIC

Periodo Segundo

Institución Educativa Sor María Juliana

Autor: Dr. Álvaro Cárdenas Orozco

Metodología MILC · Apertura ancestral · Pensamiento computacional · Triángulo Dussel-Estoicismo-Florida

Producto: Bitácora de depuración: bug encontrado + 3 hipótesis + prueba final + lección aprendida documentada.

Apertura: Depuración de lógica de control: casos esperados, contrarios y límites

Saber ancestral

El **ojo del mecánico de pueblo**: cuando un motor falla, el mecánico no cambia todo a ciegas. Busca dónde está el ruido, prueba, ajusta una pieza, prueba de nuevo. Es método disciplinado: hipótesis, prueba aislada, corrección, re-test. La depuración es el oficio del mecánico aplicado al código.

Saber contemporáneo

La **depuración (debugging)** es el arte de encontrar y corregir errores en programas. Tres tipos de casos a probar: **esperados** (lo que debe funcionar), **contrarios** (lo opuesto que también debe funcionar), **límites** (los extremos que rompen suposiciones). Sin depuración sistemática, los bugs sobreviven.

Pregunta-puente

¿Qué del ojo del mecánico de pueblo —su método disciplinado de buscar la falla— podemos llevar a la depuración de código?

Saber y saber hacer

Al terminar podrás **identificar tipos de bugs, diseñar pruebas** para casos esperados/contrarios/límites, **aislar la falla** con hipótesis verificables, **documentar** el proceso de depuración con bitácora.

Autor	Dr. Álvaro Cárdenas Orozco
DBA	Aplica método sistemático de depuración: hipótesis, prueba aislada, corrección, re-test, y documenta el proceso.
Referentes	MEN Tecnología · CSTA Standards · Modelo MILC · Dussel (filosofía de la liberación) · Marco Aurelio (obstáculo→camino) · Floridi (impacto del bien/mal informacional)
Producto final	Bitácora de depuración: bug encontrado + 3 hipótesis + prueba final + lección aprendida documentada.

Ruta MILC: así vamos a aprender

1

Escuta
Observo, escucho
y pregunto.

2

Sistematización
Pensamiento
computacional.

3

Praxis
Construyo y aplico.

4

Evaluación
Reflexión liberadora.

Fase 1 · Escuta

Escena para escuchar

Catalina programa una alarma que “se enciende cuando hay movimiento”. Pero al probar, la alarma se enciende también con el ventilador, con el aire de la ventana, con el paso de su gato. Tres bugs distintos. ¿Cómo encuentra cuál es cuál? Con depuración sistemática.

- ☐ Recuerdo una vez que algo (programa, dispositivo, app) falló y tuve que diagnosticar la causa.
- ☐ Identifico la diferencia entre bug visible (síntoma) y bug raíz (causa).
- ☐ Distingo entre prueba esperada, contraria y de límite.

Una vez tuve que diagnosticar una falla y mi proceso fue:

El error fue confundir el síntoma con la causa porque:

Lo que aprendí sobre buscar la causa raíz fue:

Saber más. La palabra *bug* viene de un evento real: en 1947, una polilla quedó atrapada en un relé del computador Mark II y causó un error. La ingeniera Grace Hopper la pegó en su bitácora con la nota “*First actual case of bug being found*”. Desde entonces “debugging” es buscar bugs.

Fase 2 · Sistematización con pensamiento computacional

Cuatro pilares aplicados al tema

El pensamiento computacional descompone la depuración en hipótesis y prueba, reconoce patrones de errores comunes, abstrae la regla *aisla, prueba, corrige, re-prueba* y diseña un algoritmo de diagnóstico sistemático.

Pilar	Aplicación al tema
1. Descomposición	Descomponer el bug: ¿cuál es el síntoma exacto? ¿en qué condiciones aparece? ¿en cuáles no?
2. Reconocimiento de patrones	Patrones de bugs: lógica invertida (NOT mal usado), umbral mal calibrado, condición de borde no contemplada, dato vacío que rompe.
3. Abstracción	Abstracción: la causa raíz es UN cambio mínimo que arregla el síntoma. Si tu corrección requiere muchos cambios, probablemente no es la raíz.
4. Algoritmo conceptual	Algoritmo: 1) reproducir el bug · 2) formar hipótesis · 3) prueba aislada · 4) confirmar o descartar · 5) corregir · 6) re-test · 7) documentar.

Anatomía de tres tipos de prueba

Caso esperado: la entrada normal que debe funcionar. Ej: para alarma de movimiento, mover la mano cerca.

Caso contrario: el opuesto, también debe funcionar. Ej: NO mover nada, la alarma NO debe activarse.

Caso de límite: valores extremos que rompen suposiciones. Ej: movimiento muy lento, muy rápido, muy lejano, valor 0 exacto.

Caso fuera de rango: ¿qué pasa si el sensor da error o no responde?

Bitácora: qué se probó, qué pasó, qué hipótesis se descartó.

Re-test: después de corregir, probar TODOS los casos de nuevo, no solo el del bug.

Errores comunes y su corrección

- Cambiar varias cosas a la vez → no sabrás cuál arregló el bug; cambia UNA cosa a la vez.
- No reproducir el bug antes de corregir → podrías “arreglar” algo que no era el problema.
- Confundir síntoma con causa → la alarma se activa por aire, pero la causa puede ser un umbral demasiado sensible.
- Probar solo casos esperados → los bugs viven en los contrarios y los límites.
- No documentar → si vuelve a aparecer en 3 meses, no recordarás cómo lo arreglaste.

Para depurar mi alarma, mi primera hipótesis sería:

Mi prueba aislada para verificarla sería:

Fase 3 · Praxis con pensamiento computacional

Construyendo el producto con los cuatro pilares

La depuración sistemática es ejercicio de pensamiento crítico. Decomponemos el bug, reconocemos patrones, abstraemos la causa raíz y seguimos un algoritmo de diagnóstico documentado.

Pilar	Aplicación al producto
Descomposición	Bitácora: 1) descripción del bug · 2) hipótesis · 3) pruebas · 4) confirmación · 5) corrección · 6) re-test · 7) lección.
Patrones	Patrones de buena depuración: hipótesis falseable, prueba aislada, cambio mínimo, documentación clara.
Abstracción	Función esencial: encontrar la causa raíz con el menor número de cambios y dejar evidencia para futuros mantenedores.
Algoritmo de armado	Pasos: reproducir → hipótesis → aislar → confirmar → corregir → re-test → documentar.

Checklist de la bitácora de depuración

- ☐ Descripción del bug (síntoma exacto, condiciones).
- ☐ 3 hipótesis distintas formuladas.
- ☐ Prueba aislada para cada hipótesis.
- ☐ Hipótesis confirmada (con evidencia).
- ☐ Corrección aplicada (cambio mínimo).
- ☐ Re-test con casos esperado, contrario y límite.
- ☐ Lección aprendida documentada.

Plantilla de trabajo

Bug: alarma de movimiento se activa con aire/ventilador.

Hipótesis 1: sensor muy sensible. Prueba: mirar valores con aire suave → si supera umbral, confirmada.

Hipótesis 2: umbral mal calibrado. Prueba: medir movimientos reales vs aire.

Hipótesis 3: código sin filtro de ruido. Prueba: ver si una sola lectura activa.

Confirmada: hipótesis 3 (sin filtro).

Corrección: requerir 3 lecturas seguidas para activar.

Re-test: mover mano (V), aire (F ahora), nada (F).

Lección: sensores deben tener filtro temporal de ruido.

Mi bitácora de depuración: bug + 3 hipótesis + corrección:

Producto de la sesión

Bitácora de depuración con bug, hipótesis y lección documentadas.

Criterios de calidad

- Bug bien descrito (síntoma + condiciones).
- 3 hipótesis falseables.
- Pruebas aisladas documentadas.
- Corrección con cambio mínimo.
- Re-test completo (esperado + contrario + límite).
- Lección aprendida.

Fase 4 · Evaluación liberadora — cinco dimensiones

Sentido de la evaluación

Evaluar no es solo revisar una nota. Es reconocer qué aprendí, qué cambió en mí, qué emociones manejé, cómo me relaciono con la ciudad y la comunidad, y qué le dejaré a quien viene detrás.

Mi reflexión liberadora en cinco dimensiones. Completa cada frase iniciada:

Dimensión	Mi respuesta (frame starter)	Algo que descubrí
1. Personal	Lo que más cambió en mí fue _____	_____
2. Emocional	La emoción más fuerte fue _____ y la regulé _____	_____
3. Ciudadana	En democracia esto importa porque _____	_____
4. Local (Cartago)	En mi barrio sería útil para _____	_____
5. Intergeneracional	A mi abuela/abuelo le diría _____	_____

Para la próxima sustentación. Vuelve a esta tabla en seis meses. Verás que las respuestas cambian — no porque lo aprendido cambie, sino porque tú cambias. La evaluación liberadora es una conversación contigo a través del tiempo.

Triángulo de pensamiento · cierre filosófico

Dussel · lente del nosotros

“La filosofía de la liberación es el pensar de los que han sido excluidos.”

Los bugs son las voces de los casos excluidos: el dato vacío que no contemplaste, el usuario que no encajaba en tu prueba esperada. La depuración es analéctica programática: dar voz a los casos olvidados.

¿Qué caso real (persona, dato, situación) no contemplaste en tu prueba inicial?

Estoicismo · Marco Aurelio · cuidado interior

“El obstáculo en el camino se vuelve el camino.”

Cada bug bien depurado es maestro: te enseña una clase de error que no volverás a cometer. La depuración es ejercicio estoico de aprender del obstáculo en lugar de evitarlo.

¿Qué bug reciente se convirtió en aprendizaje permanente sobre cómo escribir código?

Floridi · lente de la infoesfera

“El bien y el mal informacional se miden por su impacto en seres y datos.”

Un bug en software de salud no es “mal código” — es daño potencial a personas reales. Depurar bien es ética antes de ser técnica. La calidad del proceso de depuración refleja la calidad del cuidado.

Si tu programa se usara en un contexto crítico (salud, transporte, finanzas), ¿tu proceso de depuración sería suficiente?

Compromiso final

Criterio	Lo logré	En proceso	Necesito apoyo
Comprensión del tema	Explico ideas centrales con mis palabras.	Reconozco ideas pero falta precisión.	Necesito repasar conceptos base.
Pensamiento computacional	Apliqué los 4 pilares al diseñar mi producto.	Apliqué algunos pilares.	Necesito releer la fase 2.
Aplicación y producto	Mi producto cumple los criterios y se puede revisar.	Producto funcional con ajustes pendientes.	Aún en construcción.
Reflexión 5 dimensiones	Respondí cada dimensión con honestidad.	Respondí algunas con detalle.	Mis respuestas son muy generales.
Triángulo de pensamiento	Conecté las 3 voces con mi trabajo real.	Conecté al menos una.	No relaciono las voces con mi proceso.

Hoy entendí que:

Mi compromiso para la próxima sesión es:

Cita de despedida · Marco Aurelio

“El obstáculo en el camino se vuelve el camino. Lo que estorba al camino se vuelve camino.”
Cada error de hoy, bien observado, se convierte en pieza del camino que pisarás mañana.

Nombre completo:

Fecha:

Curso/grupo:

Mi firma:

Próxima sesión. Diseñaremos un sistema de alerta ambiental simulado: lectura, condición y respuesta. La depuración se vuelve disciplina de proyecto.